DSA03 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 1

Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System

Student Details	
Name	VIJAYALAKSHMI S
Register Number	192472326
Department / Class	CSE - AI
Course	DSA03 – Natural Language Processing
CO / Total Marks	CO1 / 50 Marks
Submission Date	04/11/2026

Assignment basis: The answers follow the twelve tasks and terminology in the supplied assignment brief.

Important academic note: The brief requires experiments on at least 500 de-identified records (25,000 tokens), 40 judged queries and 200 hand-annotated forms. Since those data were not supplied, all numerical evaluation values in this document are clearly marked as illustrative pilot values, not measured hospital results. Replace them with results from your own documented corpus before submission. The sample records are synthetic and contain no real patient information.

Project Overview

The proposed system prepares hospital free text for reliable search before machine learning is applied. It recognises structured clinical entities, tokenises and normalises text, validates formats with finite automata, analyses word formation with a finite-state transducer, and uses the Porter stemmer as a lightweight comparison baseline.

Design item	Specification used in this worked solution
Corpus plan	500 synthetic/de-identified records; 25,000+ tokens; discharge summaries, prescriptions, operation notes and nursing observations.
Structured patterns	12 entity classes listed in Task 2.
Morphology resources	Target lexicon: 300 stems; 20+ inflectional affixes; 15+ derivational affixes; irregular list.
Evaluation	40 clinician queries with relevance labels; 200 manually annotated word forms.
Implementation	Python standard library; NLTK PorterStemmer for Task 9 only.
Safety	Normalisation supports retrieval and must not replace clinical interpretation or alter source records.

Synthetic sample record used in worked examples

> DISCHARGE SUMMARY\nPatient ID: PT-20481 | Age: 64 years | Admission: 03/04/2025 | Discharge: 08/04/2025\nDiagnosis: multiple fractures. BP 128/82 mmHg; temperature 37.2 C.\nMedication: Paracetamol 500 mg, 1 tablet every 8 hours.\nWard: W3, Bed: 12. Patient reported pain for 2 days.

The sample is only an example for demonstrating rules. Dates are interpreted in DD/MM/YYYY format in this document.

TASK 1 – Frame the Problem (3 Marks)

1.1 Levels of linguistic analysis

Level	Meaning	Hospital-search relevance and example
Phonology	Sound patterns of spoken language.	Not central for typed records. It becomes relevant only if speech transcription is added.
Morphology	Internal structure of words and affixes.	Required: fracture, fractures, fractured and fracturing share the lemma fracture.
Syntax	How words form phrases and sentences.	Limited use: “no evidence of fracture” must not be treated like a positive fracture statement.
Semantics	Meaning of words and relations.	Important for synonym mapping: nephrectomy is related to kidney removal; renal is related to kidney, but they are not identical terms.
Pragmatics	Meaning depending on speaker, purpose and context.	Useful for interpreting “rule out fracture” or “family history of diabetes”; not fully solved by this finite-state layer.
Discourse	Meaning across multiple sentences/sections.	Useful for linking a diagnosis in one section with a later clarification, but outside the core normalisation scope.

1.2 Model families

Model family	Best-suited hospital task	Why the other families are less suitable
State machines	Validate identifiers, dates, BP and other fixed-format entities.	A probabilistic model is unnecessary for exact formats; a general rule system is less explicit for character-by-character acceptance.
Formal rule systems	Apply abbreviation expansion, case folding and morphology rules.	A state machine alone does not express all rewrite intentions conveniently; a probabilistic model may vary and needs training data.
Probabilistic models	Rank uncertain OCR corrections or resolve ambiguous abbreviations using context.	Exact state machines reject noisy strings; hand-written rules cannot cover every context and spelling variation.

1.3 Ambiguity in clinical text

- Lexical ambiguity: “cold” may mean low temperature or a common cold.
- Morphological ambiguity: “patient’s” may be patient + possessive, while “patients” is patient + plural.
- Segmentation ambiguity: “500mg” may be one quantity token or two tokens, 500 and mg.
- Abbreviation ambiguity: “MS” may refer to multiple sclerosis, morphine sulphate or another local abbreviation; context is required.

1.4 Search and resource cost

Automata perform state-space traversal: time is proportional to input length for a deterministic automaton, while memory is proportional to the number of states. Regex engines may backtrack; poorly designed nested repetition can cause excessive time. Building a clinical lexicon and judging queries consume expert effort and annotated data. The proposed finite-state core needs little run-time memory, but its manual construction and maintenance are significant.

1.5 Pipeline diagram

INPUT: discharge summaries / prescriptions / operation notes / nursing notes\n |\n v [Regex tokeniser + maximal munch] -> tokens\n |\n v [Regex entity recognisers + DFA validators] -> typed entities\n |\n v [Case / abbreviation / date / unit rewrite rules] -> normalised tokens\n |\n v [Lexicon FST + orthographic-rule transducers] -> lemma + features\n |\n v [Porter stemmer baseline] -> comparison index\n |\n v [Search index + relevance evaluation] -> ranked clinical records

Output: a searchable index containing original text references, canonical entities, tokens, lemmas and morphological features. The original record is retained unchanged.

TASK 2 – Basic Regular Expression Patterns (4 Marks)

2.1 Basic operators

Operator	Meaning	Clinical example

Concatenation	Items occur in sequence.	PT-[0-9]{5}
Disjunction |	Either alternative is accepted.	mg|mcg
Kleene star *	Zero or more repetitions.	[0-9]*
Plus +	One or more repetitions.	[0-9]+
Optional ?	Zero or one occurrence.	(kg)?
Character class	One character from a set.	[0-9]
Range	A continuous character range.	[A-Z]
Negated class	Any character except listed ones.	[^\s]+
Anchors ^ $	Beginning and end of string.	^PT-[0-9]{5}$
Counters {n,m}	Bounded repetitions.	[0-9]{2,4}

2.2 Twelve entity patterns

Entity	Regex (Python raw-string form)	Must match / must not match
Patient identifier	\\bPT-[0-9]{5}\\b	Matches PT-20481; rejects PT-2481 and PT-204810.
Admission date	\\b(?:0[1-9]|[12][0-9]|3[01])/(?:0[1-9]|1[0-2])/20[0-9]{2}\\b	Matches 03/04/2025; rejects 32/04/2025. Calendar validation is a later check.
Discharge date	\\b(?:0[1-9]|[12][0-9]|3[01])/(?:0[1-9]|1[0-2])/20[0-9]{2}\\b	Same format; rejects ISO date unless a separate ISO alternative is added.
Drug + strength	\\b[A-Za-z][A-Za-z-]*\\s+[0-9]+(?:\\.[0-9]+)?\\s?(?:mg|mcg|g|mL)\\b	Matches Paracetamol 500 mg; rejects “500 tablets”. Drug name must be checked against lexicon.
Dosage frequency	\\b(?:once|twice|thrice|every\\s+[0-9]+\\s*(?:h|hours|days)|[0-9]+\\s*times\\s+daily)\\b	Matches every 8 hours; rejects every hours.
Lab value + unit	\\b[A-Za-z][A-Za-z -]*:\\s*[0-9]+(?:\\.[0-9]+)?\\s?(?:mg/dL|g/dL|mmol/L|U/L)\\b	Matches Glucose: 110 mg/dL; rejects a value without a recognised unit.
Blood pressure	\\b(?:[6-9][0-9]|1[0-9]{2}|2[0-4][0-9])/(?:[3-9][0-9]|1[0-4][0-9])\\s?mmHg\\b	Matches 128/82 mmHg; rejects 128-82 mmHg. Range is deliberately broad.
Temperature	\\b(?:3[4-9]|4[0-2])(?:\\.[0-9])?\\s?(?:°?C|F)\\b	Matches 37.2 C; rejects 99.9 C. Clinical range validation is separate.
ICD code	\\b[A-TV-Z][0-9][0-9A-Z](?:\\.[0-9A-Z]{1,4})?\\b	Matches S72.001A; rejects 72S.
Ward and bed	\\bWard\\s+[A-Z][0-9]?\\s*,?\\s*Bed\\s*[:#]?\\s*[0-9]{1,3}\\b	Matches Ward W3, Bed 12; rejects a missing bed number.
Contact number	\\b(?:\\+91[-]?)?[6-9][0-9]{9}\\b	Matches 9876543210; rejects a 9-digit number.
Age	\\b(?:[1-9][0-9]?|1[01][0-9]|120)\\s?(?:years?|yrs?)\\b	Matches 64 years; rejects 0 years or 150 years.

Symptom duration	\\b(?:for\\s+)?[0-9]+\\s?(?:hours?|days?|weeks?|months?)\\b	Matches for 2 days; rejects “for days”.

Note: The assignment asks for at least twelve types; this solution provides thirteen patterns by treating admission and discharge dates as separate fields. Regex detects a shape, not clinical truth. A date such as 31/02/2025 passes the pattern and must be rejected by calendar-aware validation.

2.3 Greedy and non-greedy matching

Sentence: “Give 500 mg every 8 hours, then 250 mg every 12 hours.”\nGreedy: r"Give (.*) mg" -> may span from after Give to the LAST mg.\nNon-greedy:r"Give (.*?) mg" -> stops at the FIRST mg.

The greedy quantifier .* consumes as much as possible while still allowing the rest of the pattern to match. The non-greedy .*? initially consumes as little as possible and expands only as needed. For extracting each dose, a bounded pattern such as a drug/number/unit expression is safer than either broad expression.

2.4 Precedence and grouping

Intended: ^(mg|mcg)$ -> exactly “mg” OR exactly “mcg”.\nMisplaced: ^mg|mcg$ -> “starts with mg” OR “ends with mcg”.

Anchors bind to the alternatives only when grouped. Parentheses make the intended scope explicit and prevent silent acceptance of strings such as “mgXYZ”.

2.5 Error analysis (illustrative pilot)

Pattern group	TP	FP	FN	Likely error cause
Identifiers / contacts	98	1	2	OCR confuses 0/O and 1/I.
Dates	95	3	5	Ambiguous date order and impossible calendar dates.
Drug-strength / frequency	91	5	9	Unlisted brand names, spacing and misspellings.
Vitals / lab values	94	4	6	Unit variants and values split by OCR.
ICD / ward / age / duration	96	2	4	Local formats and abbreviations.

These are example counts to show how to report results, not measured results from the required corpus. In the final experiment, count each gold entity once and explain each FP/FN from the annotated records.

TASK 3 – Regex Tokenisation and Normalisation (4 Marks)

3.1 Token definition and tokenizer

A token is a clinically useful lexical or numeric unit. Hyphenated words and dotted abbreviations are kept together; decimal numbers are kept together; a number joined to a unit is one quantity token; punctuation is emitted separately only when it carries structure.

```
TOKEN_RE = re.compile(
 r"[A-Za-z]+(?:[-'][A-Za-z]+)*\.?"      # words, hyphens, apostrophes, dotted short forms
 r"|[0-9]+(?:\.[0-9]+)?(?:mg|mcg|g|mL|mmol/L|mmHg|kg|°?C)?"
 r"|[^\w\s]", re.IGNORECASE
)
```

3.2 Maximal-munch examples

Input	Whitespace split	Maximal-munch tokenisation
BP 128/82 mmHg	BP | 128/82 | mmHg	BP | 128 | / | 82 | mmHg (entity recogniser recombines BP value)
Take 500mg daily	Take | 500mg | daily	Take | 500mg | daily
follow-up post-op note	follow-up | post-op | note	follow-up | post-op | note
Temp 37.2°C	Temp | 37.2°C	Temp | 37.2°C

Maximal munch chooses the longest token matching at the current position. The tokenizer preserves "500mg" and "37.2°C" as quantities. Slash in blood pressure remains a separator so the entity parser can validate the complete structure.

3.3 Normalisation rules and abbreviation inventory

- Case folding: convert alphabetic tokens to lowercase for the search index; retain original case in the source record.
- Date canonicalisation: DD/MM/YYYY and DD-MM-YYYY are converted to YYYY-MM-DD only after the date convention is confirmed.
- Unit canonicalisation: "µg" and "mcg" → "mcg"; "milligrams" → "mg"; "C" and "°C" → "°C".

No.	Abbreviation expansion
1	BP → blood pressure
2	HR → heart rate
3	RR → respiratory rate
4	Temp → temperature
5	T → temperature (only in vitals context)
6	SOB → shortness of breath
7	N/V → nausea and vomiting
8	c/o → complains of
9	h/o → history of
10	f/u → follow-up
11	Dx → diagnosis
12	Rx → prescription / treatment
13	Tx → treatment
14	Hx → history
15	OD → once daily (context checked)
16	BD → twice daily (context checked)
17	TDS → three times daily
18	QID → four times daily
19	PRN → as needed
20	NPO → nothing by mouth
21	IV → intravenous
22	IM → intramuscular
23	PO → by mouth
24	tab → tablet
25	cap → capsule
26	WNL → within normal limits

27	CNS → central nervous system
28	UTI → urinary tract infection
29	DM → diabetes mellitus (context checked)
30	HTN → hypertension
31	ECG → electrocardiogram

3.4 Before-and-after sample record

BEFORE:\nDISCHARGE SUMMARY\nPatient ID: PT-20481 | Age: 64 years | Admission: 03/04/2025 | Discharge: 08/04/2025\nDiagnosis: Multiple fractures. BP 128/82 mmHg; Temp 37.2 C.\nMedication: Paracetamol 500 mg, 1 tab every 8 hrs.\nWard: W3, Bed: 12. Patient c/o pain for 2 days.\n\nAFTER (search representation):\ndischarge summary\npatient_id=pt-20481 | age=64 years | admission=2025-04-03 | discharge=2025-04-08\ndiagnosis: multiple fractures. blood pressure=128/82 mmHg; temperature=37.2 °C.\nmedication: paracetamol 500 mg, 1 tablet every 8 hours.\nward=w3, bed=12. patient complains of pain for 2 days.

3.5 Irreversible decisions and limits

- Case folding loses original capitalisation; acceptable for case-insensitive search, but not for preserving acronyms or names in a display copy.
- Abbreviation expansion loses the exact wording and may choose the wrong sense; store the original token and expansion together.
- Unit conversion or rounding can lose precision; retain the original numeric value and unit alongside any converted value.

A classical regular expression cannot recognise arbitrarily nested balanced parentheses. The language of balanced parentheses is not regular: a finite automaton has finite memory and cannot count an unbounded nesting depth. A stack-based pushdown automaton is needed.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence (5 Marks)

4.1 Formal definition

A deterministic finite automaton is a five-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, Σ is the input alphabet, $\delta: Q \times \Sigma \to Q$ is the transition function, q_0 is the start state and F is the set of accepting states.

4.2 Four DFA validators

For compactness, each DFA uses character classes as input symbols. Any transition not listed goes to a dead state $\perp$; $\perp$ loops to itself. This convention makes the transition table complete.

Patient ID PT- + 5 digits

Diagram (linear form): q0 -P→ q1 -T→ q2 ---→ q3 -digit→ q4 -digit→ q5 -digit→ q6 -digit→ q7 -digit→ q8*

Transition table: q0:P→q1; q1:T→q2; q2:-→q3; q3:d→q4; q4:d→q5; q5:d→q6; q6:d→q7; q7:d→q8. q8 accepts only at end.

Age 1–120 years (numeric core)

Diagram (linear form): q0 --1..9→ q1*; q1 --0..9→ q2*; q2 --0..9→ q3*; q3 only accepts values <=120 by range guard

Transition table: Numeric DFA checks one to three digits; semantic range guard rejects 000, 121–999. A following literal space + years/yrs is checked by suffix states.

Blood pressure digits/digits mmHg

Diagram (linear form): q0 --digit→ q1 --digit→ q2 --/→ q3 --digit→ q4 --digit→ q5 --space→ q6 --m→q7 --m→q8 --H→q9 --g→q10*

Transition table: q0:d→q1; q1:d→q2; q2:/→q3; q3:d→q4; q4:d→q5; q5:space→q6; q6:m→q7; q7:m→q8; q8:H→q9; q9:g→q10.

Contact number

Diagram (linear form): q0 --[6-9]→ q1 --digit→ q2 --digit→ q3 --digit→ q4 --digit→ q5 --digit→ q6 --digit→ q7 --digit→ q8 --digit→ q9 --digit→ q10*

Transition table: q0:[6-9]→q1; q1..q9:digit→next; q10 accepts at end. Optional +91 prefix is handled by a separate start branch.

Each DFA has at least six states; the identifier and contact validators explicitly use one state per required character. The age validator includes a range guard because a pure digit-count DFA alone cannot enforce the numeric range in the abbreviated table.

4.3 NFA with ε transitions – optional country code

> Language: (+91-)? followed by a 10-digit Indian mobile number.\nStates: s(start), a, b, c, d1...d10, f(accept).\nTransitions: s --ε--> d1 (no prefix); s --+--> a --9--> b --1--> c -- - --> d1;\nd1 --[6-9]--> d2; d2..d10 --[0-9]--> next; d10 is accepting.

Accepted trace for 9876543210: {s} --ε--> {d1} --9--> {d2} --8--> {d3} ... --0--> {d10}; accepted. Rejected trace for 5876543210: from s, the no-prefix branch expects [6-9], and the prefix branch expects "+"; neither consumes 5, so the live-state set becomes empty and the string is rejected.

4.4 Subset construction and minimisation

DFA subset	Input digit class	Next subset	Accept?
{s}	[6-9]	{d2}	No
{s}	other	∅	No
{d2}	[0-9]	{d3}	No
{d3}	[0-9]	{d4}	No
{d4}	[0-9]	{d5}	No
{d5}	[0-9]	{d6}	No
{d6}	[0-9]	{d7}	No
{d7}	[0-9]	{d8}	No
{d8}	[0-9]	{d9}	No
{d9}	[0-9]	{d10}	Yes
{d10}	any	∅	Yes only at end
∅	any	∅	No

The prefix branch adds intermediate subsets {a}, {b}, {c} before it joins {d1}; the no-prefix branch and prefix branch therefore converge to the same digit-validation suffix. After subset construction, equivalent

dead and accepting terminal behaviour can be merged where the end-of-input convention permits it. For this structured NFA, the minimal DFA has the same essential chain of digit-count states plus prefix states; the exact state count depends on whether the optional-prefix branch is included in the same start state and whether the dead state is counted. In the compact specification: NFA 14 states; subset DFA 15 reachable subsets; minimised DFA 14 states (including dead state).

4.5 Thompson construction and reverse equivalence

For the simple expression PT-[0-9]{5}, Thompson construction concatenates literal transitions P, T and -, then five digit transitions. Each concatenation joins fragment end to the next fragment start by ε transitions. The resulting NFA accepts exactly PT- followed by five digits. Conversely, eliminating states from the corresponding DFA using state-elimination produces the expression PT-[0-9][0-9][0-9][0-9][0-9], equivalent to PT-[0-9]{5}. Equivalence is checked by testing both recognisers against the same accepted and rejected strings.

4.6 Pumping lemma proof

Assume L = { “(”^{n} “)”^{n} : n ≥ 0 } is regular. Let p be its pumping length and choose w = “(”^{p} “)”^{p}. For any split w = xyz with |xy| ≤ p and |y| > 0, y contains only opening parentheses. Pumping down (i = 0) gives fewer opening parentheses but the same p closing parentheses, so the result is not in L. This contradicts the pumping lemma. Therefore L is not regular. This supports the Task 3 limitation: a finite automaton cannot validate arbitrary nested balanced brackets.

TASK 5 – Inflectional Morphology (3 Marks)

5.1 Terms and distinction

A morpheme is the smallest unit carrying meaning or grammatical function. A stem is the base to which an affix attaches. An affix is a bound morpheme added before or after a stem. Inflection changes grammatical features without normally changing word class or core meaning; a paradigm is the set of forms of one lexeme.

Criterion	Inflection	Derivation
Category	Usually preserves word class: fracture → fractures (noun).	May change class or create a new lexeme: infect (verb) → infection (noun).
Productivity	Highly regular and broadly applicable.	More selective and lexically restricted.
Position	Typically outside derivational morphology in English.	Often closer to the root than inflection.

5.2 Inflectional paradigms and examples

Lexeme	Noun number / verb forms	Illustrative paradigm
fracture	Noun plural	fracture, fractures
treat	Verb tense / aspect	treat, treats, treated, treating
monitor	Verb tense / aspect	monitor, monitors, monitored, monitoring
patient	Noun number / possessive	patient, patients, patient’s, patients’
severe	Adjective comparison	severe, severer, severest (often more/most severe in usage)

English inflection also includes third-person singular -s, past -ed, progressive -ing, comparative -er and superlative -est. Clinical notes often use fragments, so not every paradigm member appears in every record.

5.3 Orthographic rules and over-generation

Rule	Implementation idea	Counter-example / caution
e-deletion	Drop final e before -ing: make + ing → making.	See + ing → seeing; indiscriminate deletion is wrong.
Consonant doubling	Double final consonant for stressed CVC base: admit → admitted.	Visit → visited, not visitted; stress and syllable conditions matter.
y-to-i	Consonant + y changes to i before -es/-ed: biopsy → biopsies.	Play → plays; vowel + y does not change.
Sibilant -es	Add -es after s, x, z, ch, sh: match → matches.	Gas → gases, but a rule must avoid adding -es to unrelated endings.

5.4 Irregular forms and effect

Irregular forms must be stored because they cannot be generated reliably by the productive suffix rules: be/was/were/been; have/has/had; go/went/gone; child/children; foot/feet; tooth/teeth; diagnosis/diagnoses; analysis/analyses. The lexicon stores each surface form with its lemma and feature bundle.

Illustrative corpus calculation: suppose 25,000 tokens contain 4,800 distinct surface types and 3,900 lemmas. Type-to-lemma ratio = 4,800 / 3,900 = 1.23. If 40 judged queries contain 80 misses and inflection alone explains 24, the inflection-related miss proportion is 24/80 = 30%. These figures are examples; calculate from the final corpus and query log.

TASK 6 – Derivational Morphology (4 Marks)

6.1 Derivational affix inventory

Affix	Meaning / example	Typical category or effect
-itis	arthritis	Forms a disease noun.
-ectomy	nephrectomy	Forms a surgical-procedure noun.
-ology	cardiology	Forms a field-of-study noun.
-osis	fibrosis	Forms a condition noun.
-aemia	anaemia	Forms a blood-related condition noun.
-ation	inflammation	Often forms an action/process noun.
-able	treatable	Verb → adjective.
-ive	sedative	Often forms adjective or noun.
nephro-	nephrology	Combining form meaning kidney.
cardio-	cardiology	Combining form meaning heart.
hyper-	hypertension	Prefix meaning excessive/high.
hypo-	hypoglycaemia	Prefix meaning low/under.
re-	reoperation	Prefix meaning again.
un-	unresponsive	Prefix expressing reversal/negation.
-al	renal	Often forms adjective.
-ic	cardiac	Often forms adjective.

-ist	specialist	Forms person noun.
-ous	infectious	Forms adjective.
-less	painless	Forms adjective meaning without.
-ment	management	Often forms action/result noun.

6.2 Six derivational families and order

Root / family	Family examples	Order and impossible order
nephr-	nephron, renal, nephritis, nephrectomy, nephrology	nephr + -itis; nephro- + -logy. English suffix stacking is constrained; *-ectomy + nephro in reverse order is not the intended construction.
cardi-	cardiac, cardiology, cardiomyopathy	cardio- + myo- + -pathy; *-logy + cardio- is not a valid order.
infect	infect, infection, infectious, reinfection	re- + infection; *-tion + infect as a suffix-first form is impossible.
fracture	fracture, fractures, fractured, fracturing	Inflection attaches after the lexeme; *fracture + -ing + -ed is not a normal single form.
operate	operate, operation, operative, reoperation	re- + operation; *-ation + re + operate is not a surface word-building order.
respond	respond, response, responsive, unresponsive	un- + responsive; *-ive + un + respond is not the standard order.

6.3 Non-compositionality and search implications

A derived medical term may have a specialised meaning not fully predictable from its parts. “Nephrectomy” is specifically surgical removal of a kidney; “nephrology” is the medical specialty, not a procedure. A stemmer that strips endings and conflates both may return misleading records. “Inflammation” and “inflammatory” are related and often useful together, but “sedative” and “sedation” are not interchangeable in every clinical query.

6.4 Productivity and conflation decision

Suffixes such as -ed, -ing and -s are highly productive inflectional endings. Derivational -itis, -ectomy and -ology are productive within medical terminology but attach only to appropriate roots. Some forms are fossilised or lexically restricted, such as anaemia and nephron. A final empirical report should count distinct affix-bearing types and total tokens per affix. Because the supplied file contains no corpus, counts cannot honestly be claimed here.

Conflation helps	Conflation can mislead
fracture / fractures / fractured: retrieve all inflectional forms.	nephrectomy / nephrology: both contain nephro- but denote different concepts.
infect / infection / infectious: broaden a query when the clinical intent is general.	sedative / sedation: a drug class and a process are not identical.

TASK 7 – Finite-State Morphological Parsing (5 Marks)

7.1 FST definition

A finite-state transducer (FST) is a finite-state machine with an input tape and an output tape. In analysis, it reads a surface word and outputs a lexical stem plus grammatical features. In generation, it reads the lexical representation and produces the surface word. The same relation can be traversed in opposite directions.

Surface tape: f r a c t u r e s\nLexical tape: f r a c t u r e +N +PL\nAnalysis: fractures → fracture +N +PL

7.2 Lexicon and continuation classes

Class	Example stems	Allowed continuation
NOUN	fracture, patient, diagnosis, kidney, infection	+N → +SG / +PL / +POSS; selected derivational suffixes
VERB	treat, monitor, admit, infect, respond	+V → +PRS / +3SG / +PAST / +PROG
ADJECTIVE	severe, renal, cardiac, responsive	+ADJ → +COMP / +SUPER; selected derivational prefixes
MEDICAL ROOT	nephr-, cardi-, gastro-, derm-	Combining forms followed by licensed roots/suffixes

The production lexicon is planned to contain at least 300 stems. The inventory should include common clinical nouns, verbs, adjectives, abbreviations and medical combining forms. At least 20 inflectional affixes/features and 15 derivational affixes are documented in the resource file; irregulars are stored as explicit mappings.

7.3 Hand traces

Surface form	State path (simplified)	Lexical analysis
fractures	NOUN → plural suffix path	fracture +N +PL
fractured	VERB/ADJ path → past/participle	fracture +V +PAST (context may also allow adjective)
treating	VERB → progressive suffix	treat +V +PROG
patients	NOUN → plural suffix	patient +N +PL
unresponsive	ADJ → negative prefix + adjective	un- + responsive +ADJ

7.4 Generation direction

The same transducer maps fracture +N +PL to fractures, treat +V +PROG to treating, and patient +N +PL to patients. Parsing and generation are inverse traversals of the same lexical-to-surface relation; separate machines are not required.

7.5 Ambiguity

- "treated" can be analysed as treat +V +PAST or treat +V +PASTPART, depending on syntactic context.
- "patients" can be patient +N +PL; in a specialised tokenisation it may also be patient +N +POSS if an apostrophe was lost by OCR. The analyser should retain both candidates only when the observed spelling and noise model justify them.

The FST returns all legal analyses. A downstream ranker can use part of speech, surrounding words, section type and clinical terminology to select or rank candidates. It must preserve alternatives when confidence is low.

TASK 8 – Orthographic Rules and Two-Level Morphology (4 Marks)

8.1 Why plain concatenation fails

- make + ing gives *makeing; correct surface form is making.
- admit + ed gives *admited; correct surface form is admitted.
- biopsy + es gives *biopsyes; correct surface form is biopsies.

8.2 Rewrite rules

Rule	Rewrite notation	Context / example
Final e deletion	e → 0 / _ +ing	make + ing → making; only when licensed by the stem class.
Consonant doubling	C → CC / stressed V C _ +ed	admit + ed → admitted; lexical/stress condition needed.
y-to-i	y → i / C _ +es	biopsy + es → biopsies.
Sibilant plural	0 → e / {s,x,z,ch,sh} _ +s	match + s → matches.
Past -ed allomorph	e → 0 / _ +ed	treat + ed → treated (no extra e).

8.3 Cascade and two-level morphology

Lexical input → Lexicon FST → e-deletion transducer → y-to-i transducer\n → consonant-doubling transducer → suffix realisation → surface form

The order matters: the lexicon first identifies the morpheme boundary; spelling rules then operate at that boundary. Two-level morphology represents lexical and surface symbols as parallel correspondences, allowing constraints to apply simultaneously rather than as a sequence of rewrite passes. A cascade is intuitive and easy to debug; a two-level system avoids some order-dependence and can represent interacting constraints more consistently.

8.4 Test set (illustrative)

Lexical form	Expected surface	Result
make + V + PROG	making	Pass
treat + V + PAST	treated	Pass
admit + V + PAST	admitted	Pass
biopsy + N + PL	biopsies	Pass
match + N + PL	matches	Pass
study + N + PL	studies	Pass
play + V + 3SG	plays	Pass
see + V + PROG	seeing	Pass
use + V + PROG	using	Pass
visit + V + PAST	visited	Pass
stop + V + PAST	stopped	Pass
plan + V + PAST	planned	Pass

fix + N + PL	fixes	Pass
box + N + PL	boxes	Pass
wash + N + PL	washes	Pass
diagnosis + N + PL	diagnoses	Lexicon exception
analysis + N + PL	analyses	Lexicon exception
child + N + PL	children	Irregular lexicon
go + V + PAST	went	Irregular lexicon
be + V + PAST	was/were	Irregular lexicon

For the 20-form illustrative test, 20/20 expected outputs are covered by a combination of productive rules and explicit irregular entries. A genuine evaluation must record actual failures, not assume all test cases pass.

TASK 9 – The Porter Stemmer (5 Marks)

9.1 Definitions and conditions

Porter stemming is a suffix-stripping algorithm for English. A consonant is a letter that is not a, e, i, o or u; y is treated contextually (as a consonant at the start and otherwise the opposite of the preceding letter). The measure m counts vowel-consonant sequences in the stem pattern [C](VC)^m[V].

- *v: the stem contains a vowel.
- *d: the stem ends with a double consonant.
- *o: the stem ends in consonant-vowel-consonant, where the final consonant is not w, x or y.

9.2 Measure examples

Word	Stem pattern (simplified)	m (illustrative)
fracture	fr a ct u r e	2
patients	p a t i e n t s	2
treated	tr ea t ed	2
monitoring	m o n i t o r i n g	3
renal	r e n a l	2
infection	i n f e c t i o n	3

The exact m value is computed after applying Porter's contextual vowel/consonant definition and removing the suffix under consideration. The table is a teaching illustration; use the code in the appendix to verify exact step conditions.

9.3 Five Porter steps – hand-style traces

Input	Step 1 (plurals / -ed / -ing)	Step 2–3 (derivational)	Step 4–5	Final stem (typical)
fractures	fractures → fracture	No listed derivational replacement	Final e retained	fracture
patients	patients → patient	No change	No change	patient
treated	treated → treat	No change	No change	treat
monitoring	monitoring → monitor	No change	No change	monitor

relational	relational → relation	relation → relate (Step 2)	relate → relat (Step 4/5 conditions)	relat
infection	infection	No guaranteed clinical lemma recovery	Suffix stripping may leave a truncated form	infect

Porter is a general-purpose suffix stemmer, not a clinical lemmatiser. It can produce non-words such as "relat" and does not know that a clinical term should map to a dictionary lemma.

9.4 Stemming versus lemmatisation

Measure	Illustrative 200-form annotation comparison
Agreement	142/200 = 71%
Divergence	58/200 = 29%
Typical agreement	patients → patient; treated → treat
Typical divergence	fractures may be reduced differently depending on suffix rules; diagnoses requires irregular/lexicon handling; nephrectomy should not be reduced to an unsafe generic stem.

9.5 Error types and retrieval experiment

- Over-stemming: two distinct words are reduced to the same stem, creating misleading matches; this is dangerous for related but non-equivalent clinical concepts.
- Under-stemming: related forms are not reduced to the same stem, so a search misses relevant records.

Indexing strategy	Precision	Recall	F1	Interpretation (illustrative)
No stemming	0.94	0.72	0.815	High precision, misses inflected variants.
Porter stemmer	0.88	0.86	0.870	Better recall, some over-stemming.
Full morphological analysis	0.93	0.91	0.920	Balances recall and precision if lexicon is maintained.

F1 = 2 × precision × recall / (precision + recall). The figures are illustrative only. Report the 40-query experiment using identical relevance judgements for all three indexing conditions.

TASK 10 – Implement and Test the Pipeline (3 Marks)

10.1 Python implementation

The following compact implementation demonstrates the core operations. It uses Python standard-library regular expressions and the NLTK Porter stemmer. It is a reproducible starting point; the complete 300-stem lexicon, full FST continuation classes and corpus loader must be populated with the project data.

```
import re
from collections import defaultdict
try:
    from nltk.stem import PorterStemmer
    porter = PorterStemmer()
except ImportError:
```

```
    porter = None

TOKEN_RE = re.compile(
    r"[A-Za-z]+(?:[-'][A-Za-z]+)*\.?|"
    r"[0-9]+(?:\.[0-9]+)?(?:mg|mcg|g|mL|mmol/L|mmHg|kg|°?C)?|"
    r"[^\w\s]", re.I)

PATTERNS = {
 "patient_id": re.compile(r"\bPT-[0-9]{5}\b"),
 "date": re.compile(r"\b(?:0[1-9]|[12][0-9]|3[01])/(?:0[1-9]|1[0-2])/20[0-9]{2}\b"),
 "drug_strength": re.compile(r"\b[A-Za-z][A-Za-z-]*\s+[0-9]+(?:\.[0-9]+)?\s?(?:mg|mcg|g|mL)\b",
re.I),
 "frequency": re.compile(r"\b(?:once|twice|thrice|every\s+[0-9]+\s*(?:h|hours|days)|[0-
9]+\s*times\s+daily)\b", re.I),
 "bp": re.compile(r"\b[0-9]{2,3}/[0-9]{2,3}\s?mmHg\b", re.I),
 "temperature": re.compile(r"\b[0-9]{2}(?:\.[0-9])?\s?°?C\b", re.I),
 "contact": re.compile(r"\b(?:\+91[- ]?)?[6-9][0-9]{9}\b"),
 "age": re.compile(r"\b(?:[1-9][0-9]?|1[01][0-9]|120)\s?(?:years?|yrs?)\b", re.I),
 "duration": re.compile(r"\b(?:for\s+)?[0-9]+\s?(?:hours?|days?|weeks?|months?)\b", re.I)
}
ABBR = {"bp":"blood pressure", "hr":"heart rate", "temp":"temperature",
        "sob":"shortness of breath", "prn":"as needed", "iv":"intravenous",
        "im":"intramuscular", "po":"by mouth", "dx":"diagnosis",
        "hx":"history", "f/u":"follow-up", "n/v":"nausea and vomiting"}

def tokenize(text):
    return TOKEN_RE.findall(text)

def normalise(text):
    text = text.lower()
    for short, full in sorted(ABBR.items(), key=lambda x: -len(x[0])):
        text = re.sub(r"(?<!\w)" + re.escape(short) + r"(?!\w)", full, text)
    text = re.sub(r"\b([0-3][0-9])/([01][0-9])/(20[0-9]{2})\b",
                  r"\3-\2-\1", text)
    text = re.sub(r"\bmilligrams?\b", "mg", text)
    text = re.sub(r"\bmcg\b", "mcg", text)
    return text

def recognise(text):
    return {name: pat.findall(text) for name, pat in PATTERNS.items()}

# A simple lexicon-backed inflectional analyser (illustrative FST relation).
LEXICON = {"fractures": ("fracture", "+N +PL"),
           "patients": ("patient", "+N +PL"),
```

```
        "treated": ("treat", "+V +PAST"),
        "treating": ("treat", "+V +PROG"),
        "diagnoses": ("diagnosis", "+N +PL")}

def analyse(word):
    return LEXICON.get(word.lower(), None)

def generate(lemma, features):
    for surface, analysis in LEXICON.items():
        if analysis == (lemma, features):
            return surface
    return None

def run_pipeline(record):
    tokens = tokenize(record)
    clean = normalise(record)
    entities = recognise(record)
    analyses = {t: analyse(t) for t in tokens if analyse(t)}
    stems = {t: porter.stem(t) for t in tokens if porter}
    return tokens, clean, entities, analyses, stems
```

10.2 Test procedure and sample output

Test procedure:\n1. Load the documented corpus and retain a unique record ID.\n2. Run tokenizer, entity patterns, validators, normaliser, FST analyser and Porter stemmer.\n3. Compare recognised entities and word analyses against hand-labelled gold annotations.\n4. Record TP, FP, FN, rejected items and wall-clock run-time for each stage.\n5. Repeat with the same query set for all three retrieval conditions.

Stage	Items processed (sample run)	Matched / analysed	Rejected / unmatched	Runtime
Tokenizer	25,000 tokens	25,000 emitted tokens	0 token positions lost	0.08 s
Entity patterns	500 records	1,850 candidate entities	120 candidates rejected	0.04 s
DFA validators	1,850 candidates	1,730 accepted	120 rejected	0.02 s
Morphology analyser	200 annotated forms	176 analysed	24 unknown / ambiguous	0.01 s
Porter stemmer	25,000 tokens	24,600 alphabetic tokens stemmed	400 non-word / symbol tokens skipped	0.03 s

All stage counts and times above are illustrative placeholders, not a claim that the supplied 500-record corpus was executed. Replace them with measured values. Runtime depends on hardware, Python version and corpus size.

TASK 11 – Compare, Recommend and Defend the Design (2 Marks)

Strategy	Retrieval quality	Coverage / runtime	Maintenance
No normalisation	Precise exact-form matches; poor recall for variants.	Fastest; only exact forms indexed.	Low maintenance.
Porter stemming	Improves recall; may over-stem clinical vocabulary.	Very fast; broad English coverage without a lexicon.	Low maintenance; errors harder to control.
Finite-state morphology	Best control over clinical lemmas and features when lexicon is complete.	High coverage for known vocabulary; unknown words remain a weakness.	Higher initial and ongoing expert effort.

Recommendation: deploy a hybrid design. Use regex and DFA for high-confidence structured fields; use finite-state morphological analysis for known clinical vocabulary; retain Porter stemming only as a low-confidence fallback for unknown English words. Always store the original string and provenance. Search results should distinguish exact, inflectional and broader derivational matches. This avoids maximising recall at the cost of clinically misleading conflation.

Manual effort is concentrated in pattern design, lexicon construction, affix licensing, irregular-form collection and annotation. A statistical or learned approach becomes more economical when the vocabulary and writing styles change faster than experts can maintain rules, and when enough representative labelled data exist. The finite-state layer remains valuable for exact formats and explainable constraints.

TASK 12 – Day-4 Design Challenge (8 Marks)

A. Impact Analysis (2 Marks)

The new input includes OCR character substitutions (0/O, 1/l, rn/m), line-break hyphenation, missing diacritics, at least 20% out-of-lexicon or misspelt tokens, and Romanised regional-language words mixed with English. The original design assumed clean typed English, a mostly closed lexicon and exact character matches. Those assumptions no longer hold.

- Regex patterns can fail when PT-20481 is read as PT-2O481 or a unit is split across lines.
- Automata reject corrupted strings rather than expressing partial confidence.
- The English morphology FST cannot analyse regional-language inflections or unseen stems.
- A single token may contain English and regional-language morphology, so sentence-level language identification is insufficient.

If 20% of tokens are noisy or out of lexicon, analyser coverage cannot exceed 80% on those tokens without correction or lexicon extension. Pattern precision may also fall if fuzzy correction is too permissive. Actual degradation must be measured against a labelled noisy sample.

B. Evaluation of Original Design (2 Marks)

New record	Original pipeline trace	First failure
OCR: “PT-2O481 Temp 37.2 C”	ID pattern rejects letter O in numeric position; temperature may match after spacing normalisation.	Identifier DFA rejects O where a digit is required.

Code-mixed: “patient ku fever irundhuchu, paracetamol 500mg kuduthen”	English tokens patient, fever and paracetamol may be recognised; ku, irundhuchu and kuduthen are unknown.	English-only FST has no analyses for regional tokens.
Both: “PT-2O481; patient ku fever irundhuchu; 500 rn g”	Identifier fails; tokeniser splits corrupted unit; regional words remain unknown.	OCR correction and token repair are absent.

Illustrative noisy-sample comparison: clean pattern precision/recall 0.96/0.94; noisy original pipeline 0.78/0.70. Clean analyser coverage 0.88; noisy coverage 0.64. Clean retrieval F1 0.92; noisy F1 0.68. These figures are illustrative and must be replaced by a controlled test. Weighted automata and edit-distance matching can rank likely corrections; character n-gram language identification can flag token language.

C. Adaptation or Redesign (2 Marks)

Decision: MODIFY the original finite-state pipeline rather than discard it. Preserve exact matching for clean inputs and insert a confidence-aware repair and language-identification layer before morphology.

> Raw OCR / typed note\n ↓\nLine repair + Unicode normalisation + OCR confusion candidates\n ↓\nWeighted edit-distance matcher (candidate corrections + confidence)\n ↓\nToken-level language identification (English / regional / mixed / unknown)\n ↓\nRegex + DFA exact validators (high-confidence fields)\n ↓\nEnglish clinical FST + expanded regional-language lexicon/FST\n ↓\nUnknown-token fallback (Porter for English-like tokens; preserve others)\n ↓\nSearch index with original token, corrected candidate and confidence

- Patterns: add controlled alternatives for common OCR confusions only in numeric fields and validate the corrected value against context.
- Automata: retain deterministic validators for clean strings; use a weighted edit-distance front end to propose repairs rather than making the DFA itself fuzzy.
- Lexicon: add regional stems, common transliterations, local abbreviations and frequent OCR variants with source labels.
- Rule cascade: add line-end hyphen repair and Unicode normalisation before spelling rules; retain original text for audit.

Expected effect: coverage should increase, but precision can decrease if candidate correction is too permissive. Use confidence thresholds and human review for identifiers, medication strengths and dates. Runtime increases because multiple correction candidates are evaluated. Annotation effort increases because code-mixed and OCR examples must be labelled.

D. Evidence-Based Defence (2 Marks)

Modification is justified because the original finite-state layer is reliable for clean, known patterns but cannot accept noisy spellings or unseen regional morphology. A weighted correction stage handles uncertainty; language identification routes tokens to the appropriate analyser; the original DFA remains responsible for exact validation after correction. This division preserves explainability and reduces the risk of accepting a wrong patient ID or drug dose.

Remaining limitations: a finite-state system cannot infer unrestricted meaning, resolve all clinical context, understand negation across long discourse, or guarantee the correct expansion of an ambiguous

abbreviation. OCR correction can generate plausible but incorrect alternatives. Code-mixed regional language requires sustained expert lexicon work. High-risk fields must be confirmed against the source record and never silently overwritten.

Conclusion

The recommended hospital search layer combines regular expressions, deterministic and non-deterministic automata, a finite-state morphological transducer and Porter stemming. Regex and automata provide precise recognition of structured strings; morphology connects surface variants to lemmas and features; Porter offers a fast baseline. Under OCR and code-mixed input, the best choice is a modified hybrid pipeline with weighted correction, token-level language identification and explicit confidence handling. The original record must always be preserved.

References

- Jurafsky, D. and Martin, J. H. Speech and Language Processing. Draft textbook, chapters on regular expressions, finite-state morphology and stemming.
- Porter, M. F. (1980). An algorithm for suffix stripping. Program, 14(3), 130–137.
- Koskenniemi, K. (1983). Two-Level Morphology: A General Computational Model for Word-Form Recognition and Production. University of Helsinki.
- Python Software Foundation. Python re — Regular expression operations. https://docs.python.org/3/library/re.html
- NLTK Project. NLTK documentation: PorterStemmer. https://www.nltk.org/api/nltk.stem.PorterStemmer.html

DETAILED PROJECT METHODOLOGY AND CORPUS PREPARATION

The corpus should be prepared before pattern evaluation begins. Each record receives a non-identifying record key, record type, source type and token count. The four minimum record types are discharge summaries, prescriptions, operation notes and nursing observations. The project should use synthetic text or data that has been properly de-identified and approved for academic use.

To reach the required minimum, prepare at least 500 records and verify that their combined token count is at least 25,000. A useful sampling plan is to distribute records across the four types rather than allowing one type to dominate. For example, 125 records per type gives a balanced starting point. The actual distribution and token counts must be reported.

Keep three versions of every record: the immutable original, a tokenised representation and a normalised search representation. The normalised representation should retain offsets or token references so that a search result can highlight the original text. Never overwrite an original clinical statement with an abbreviation expansion or OCR correction.

Create a data dictionary for every field. Define the date convention, identifier format, accepted units, abbreviation policy, language labels and annotation conventions. If the corpus contains more than one date convention, record the convention at record level rather than guessing from the date string.

Before testing, divide the corpus into development and evaluation subsets. Develop patterns using the development subset, then evaluate once on the held-out subset. This reduces the risk of reporting artificially good results after repeatedly changing patterns to fit the same examples.

TASK 1 – AMBIGUITY AND RESOURCE ANALYSIS: EXTENDED DISCUSSION

Ambiguity is central to natural language processing because the same written form can support more than one interpretation, and different forms can express the same underlying concept. In clinical records, ambiguity is not merely a linguistic inconvenience: it can change which records are retrieved and how a clinician interprets a result.

Lexical ambiguity occurs when one word has more than one sense. “Cold” can describe low temperature or an upper respiratory illness. Abbreviation ambiguity is especially common: “MS” may refer to multiple sclerosis or another local abbreviation. A safe system should use section, neighbouring terms and a maintained abbreviation list before expanding it.

Morphological ambiguity occurs when a surface form supports different grammatical analyses. “Treated” may be a past-tense verb or a past participle. Segmentation ambiguity occurs when boundaries are uncertain, as with “500mg”, “N/V” or OCR output that inserts spaces inside a unit. These cases motivate keeping multiple candidate analyses when certainty is low.

State-space search occurs when an automaton follows transitions through its states. A DFA has one active state at a time; an NFA may have a set of active states. Regex engines may use backtracking to reconsider earlier choices. Time and memory use depend on the expression and engine, so patterns should avoid unnecessary nested ambiguous repetition.

Expert effort is consumed by defining valid formats, collecting clinical vocabulary, deciding abbreviation senses, annotating gold examples and reviewing errors. A realistic project report should distinguish machine runtime from human construction and maintenance cost.

TASK 2 – ENTITY PATTERN SPECIFICATION AND SAFETY

Every entity pattern needs a positive specification and a negative specification. The positive specification states the exact forms that should be matched. The negative specification states near-miss forms that must be rejected. This prevents a pattern from being judged only by examples that it already matches.

Patient identifiers should be anchored to the complete field and should not be accepted as substrings of longer strings. Dates should be checked in two stages: first, a regular expression checks the shape; second, a date parser checks whether the day exists in the specified month and year. This separates lexical recognition from calendar validity.

Drug-strength recognition should be treated as candidate extraction, not medication verification. The expression can recognise a name followed by a number and a unit, but the drug name must be checked against an approved vocabulary. Strengths may use decimal values and alternative units. The system should preserve both the number and its unit.

Blood pressure and laboratory values also need semantic range checks. A value may have the right punctuation and unit but still be implausible or incorrectly transcribed. The regex should identify the candidate span; a separate validator should check numeric ranges and units. Such checks should flag anomalies rather than silently alter values.

An error log should store the record key, character offsets, pattern name, extracted text, gold label, predicted label and error category. This makes false positives and false negatives traceable and allows patterns to be improved systematically.

TASK 3 – TOKENISATION EDGE CASES

Tokenisation decisions should be consistent across all record types. A tokenizer that splits “post-operative” but preserves “follow-up” inconsistently will create avoidable variation in the index. The project should specify whether hyphens are retained, whether apostrophes are retained and whether dotted abbreviations are treated as one token.

Decimal values must not be split at the decimal point. A temperature of 37.2 °C contains a numeric value and a unit; the representation may keep them as one token or as two linked subfields, but the decision must be documented. Similar care is needed for values such as 5.5 mg/mL and 120/80 mmHg.

Maximal munch is useful where a longer token should take priority over shorter valid pieces. For example, a multi-character unit such as mmol/L should be recognised as a complete unit rather than broken into mmol, slash and L. The implementation should order alternatives carefully and test overlapping token patterns.

Normalisation should be reversible wherever possible. Case folding can be handled by retaining the original token. Abbreviation expansion can store both abbreviation and expansion. Unit conversion should store original value, original unit, converted value and conversion rule. These paired representations support auditing and prevent loss of information.

Regular expressions cannot express arbitrary nested balanced parentheses with finite memory. A clinical note may contain nested explanatory text. The regex layer can identify ordinary punctuation and shallow patterns, but a parser with a stack is needed for unbounded nesting.

TASK 4 – DFA DESIGN, COMPLETENESS AND MINIMISATION

A DFA transition table should explicitly define the next state for every input class from every state. In a compact report, invalid transitions may be grouped into a dead state, but the dead-state behaviour must be stated. A complete DFA has a defined transition for every state and every symbol in its alphabet.

For a patient identifier, the alphabet can be partitioned into P, T, hyphen, digit and other. This avoids listing every possible character separately. The transition table follows the required order. Any unexpected character sends the machine to the dead state, which loops to itself.

An automaton for age requires care. A pattern may accept one to three digits, but the valid age range is not the same as the number of digits. The values 000 or 150 have the correct general digit shape but may be invalid under the chosen policy. A numeric range check is often clearer than adding many DFA branches.

Subset construction begins with the epsilon closure of the NFA start state. For each subset and input symbol, collect all destinations and then take epsilon closure again. Continue until no new subsets are discovered. The subset table should show the complete reachable state set, not only the path used by one sample string.

Minimisation partitions states into accepting and non-accepting groups and repeatedly refines the partitions when transitions lead to different groups. Report the number of reachable states before minimisation and after minimisation, including whether the dead state is counted. The exact count must come from the actual automaton.

TASK 4 – THOMPSON CONSTRUCTION AND PUMPING LEMMA

Thompson's construction creates an epsilon-NFA from a regular expression. A symbol creates a two-state fragment. Concatenation connects the end of one fragment to the beginning of the next. Union introduces a new start and final state with epsilon branches. Kleene star adds epsilon transitions that allow zero or repeated traversals.

For the identifier expression PT-[0-9]{5}, the construction concatenates the literal P, literal T, hyphen and five digit-class fragments. The resulting automaton accepts exactly the strings that have this sequence. Testing should include a valid identifier, a short identifier, a long identifier and a string containing a letter in a digit position.

State elimination can derive a regular expression from an automaton. When a state is removed, incoming and outgoing paths are combined using concatenation, union and star. The resulting expression should be tested against the automaton to verify equivalence on a broad set of strings.

The pumping lemma states that if a language is regular, there is a pumping length p such that any sufficiently long string can be divided into xyz with |xy| no greater than p, |y| greater than zero, and xy^i z in the language for every i at least zero.

For balanced parentheses, choose a string with p opening parentheses followed by p closing parentheses. Since y must lie within the first p symbols, it consists only of opening parentheses. Pumping y down leaves too few opening parentheses, so the string is unbalanced. This contradiction proves that the language is not regular.

TASK 5 – INFLECTIONAL PARADIGMS AND MEASUREMENT

An inflectional paradigm records the forms a lexeme can take for grammatical features. For a noun, the main contrast may be singular and plural: patient/patients and fracture/fractures. For a verb, the paradigm may include base, third-person singular, past and progressive: treat, treats, treated and treating.

English spelling rules are conditional. Final-e deletion applies in selected cases before vowel-initial suffixes. Consonant doubling depends on stress and syllable structure. The y-to-i rule applies after a consonant but not after a vowel. Sibilant endings often take -es. A robust implementation must include exceptions and must not apply a rule merely because the final letters look similar.

Irregular forms require lexical entries because their surface forms cannot be generated by the productive rules. The entries should map every observed form to the same lemma and the appropriate feature bundle. For example, went maps to go with a past-tense feature, while children maps to child with plural number.

Type-to-lemma ratio should be calculated from the complete annotated corpus. Count unique surface types after the agreed tokenisation policy. Then map each type to a lemma and count unique lemmas. Divide the first count by the second. Clearly state whether punctuation, numbers and abbreviations are excluded.

To quantify retrieval benefit, identify query misses that would have been found if inflectional variants were conflated. Divide the number of such misses by all relevant misses. This isolates the contribution of inflection rather than attributing every improvement to the entire normalisation system.

TASK 6 – DERIVATIONAL MORPHOLOGY AND CLINICAL MEANING

Derivational morphology is harder than inflection because affixes are more selective and can change word class or specialised meaning. A productive inflectional ending can apply to many nouns or verbs, while a medical suffix may attach only to particular roots and create a conventional term.

The resource should record at least fifteen derivational affixes with examples and category information. Useful entries include -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un-, -al and -ic. Additional entries can include cardio-, gastro-, -ist, -ous, -less and -ment.

Affix productivity can be estimated from corpus evidence. Type count is the number of distinct words containing the affix; token count is the total number of occurrences. A high type count across many roots suggests greater productivity. A low count in a few fixed terms suggests a restricted or fossilised pattern.

Semantic non-compositionality means the whole word's meaning is not fully predictable from the meaning of its pieces. Medical terminology contains conventional meanings that must be represented in a lexicon. The search system should therefore distinguish morphological relatedness from clinical synonymy.

Conflation policy should be conservative. Inflectional variants of the same lexeme are generally safe to group. Derivational relatives should be grouped only when a controlled clinical vocabulary or clinician-approved relation supports it. The search interface can label broader related-term matches separately.

TASK 7 – FST IMPLEMENTATION AND AMBIGUITY

A finite-state transducer can be represented as a graph whose arcs carry an input symbol and an output symbol. Some arcs may consume a surface character while emitting no lexical symbol; others may emit a feature tag at a morpheme boundary. The graph is designed so that valid paths encode valid analyses.

Continuation classes reduce duplication. A noun class can share plural and possessive endings; a verb class can share tense and aspect endings; an adjective class can share comparative and superlative paths. Individual stems point to the continuation class that is appropriate for their category and spelling behaviour.

Parsing and generation use the same relation in opposite directions. If the transducer maps fracture +N +PL to fractures, reversing the relation maps fractures back to fracture +N +PL. The implementation should demonstrate both directions with the same stored transitions.

Ambiguity should not be hidden. A surface form such as treated may have multiple grammatical analyses. The analyser can return all candidates and a downstream component can rank them using sentence context, section type or neighbouring words. If the evidence is insufficient, preserve the alternatives.

Coverage and accuracy are different. Coverage measures how many tokens receive at least one analysis. Accuracy measures how many returned analyses are correct. A system can have high coverage but over-generate incorrect analyses. Both metrics should be reported.

TASK 8 – ORTHOGRAPHIC RULES AND CASCADE DESIGN

Orthographic rules connect lexical morphemes to surface spelling. The lexicon identifies the stem and suffix, while rewrite rules handle spelling changes at the boundary. This avoids listing every inflected surface form separately and makes the system easier to extend.

Rule notation such as a → b / c __ d means that a is rewritten as b when it occurs after context c and before context d. The symbol 0 can represent insertion or deletion. For example, final e may be deleted before -ing; y may become i before -es after a consonant; and a consonant may double under the appropriate stress conditions.

A sequential cascade applies transducers in a defined order. This is easy to inspect but can be sensitive to order. A two-level model represents lexical-surface correspondences in parallel and can express interacting constraints without requiring the same sequence of rewrite stages.

Testing should cover regular forms, exception forms and deliberately incorrect forms. The report should show the lexical input, expected surface output, actual surface output and whether the result passed. Failures should be classified as missing lexicon entry, incorrect continuation class, incorrect rule context, rule-order problem or irregular exception.

Do not report a perfect test result unless the tests were actually executed. If a form is handled by an explicit irregular entry rather than a productive rule, label it accordingly. This distinction is important when discussing generalisation.

TASK 9 – PORTER STEMMER TECHNICAL NOTES

The Porter algorithm is a sequence of suffix-reduction stages. It uses the measure m to estimate the structure of the stem. In a simplified representation, m counts the number of vowel-consonant transitions in the stem pattern. The algorithm also checks whether a stem contains a vowel, ends in a double consonant or has a consonant-vowel-consonant ending.

Step 1 handles plurals and verb endings. Later steps handle derivational suffixes and selected final-letter conditions. The exact order is important because the output of one stage becomes the input to the next. The hand trace should include each intermediate form and the condition that allowed the rule.

Porter stemming is intentionally not a dictionary lookup. It may return a truncated form that is not a valid English word. This is acceptable in some general information-retrieval settings but requires caution in medical search, where closely related terms may have different clinical meanings.

Use NLTK's PorterStemmer for reproducible implementation and record the NLTK version. Test the same 200 annotated word forms with both Porter stemming and the morphological analyser. Create a comparison table showing agreement, divergence, over-stemming and under-stemming.

Run the same 40 queries against each index. Do not change query wording or relevance judgements between conditions. Report precision, recall and F1. Discuss why a method with higher recall may still be unsuitable if it returns clinically misleading matches.

TASK 10 – IMPLEMENTATION, TEST PLAN AND VISUALISATION

The software should be divided into small functions so that each stage can be tested independently. A tokenizer function returns tokens. A normaliser function applies case folding and approved substitutions. An entity recogniser returns candidate spans. A validator checks each candidate. A morphology function returns lemma and feature analyses. A Porter wrapper returns stems.

Unit tests should cover valid and invalid examples for every pattern and automaton. Include boundary cases such as an identifier that is one digit too short, an invalid date, a blood pressure value missing its unit, an age outside the accepted range and a drug strength with an unsupported unit.

Integration tests should run a complete sample record through every stage and compare the result with a manually calculated expected output. The output should include original text, tokens, extracted entities, normalised tokens, morphological analyses and stems. This makes errors easier to locate.

Measure runtime using the same machine and corpus for each method. Report record count, token count and elapsed time. A graph of runtime against record length can show whether processing time grows approximately linearly. A graph of analysis coverage against lexicon size can show the effect of adding stems.

Every figure and table should state whether it contains actual measurements or illustrative values. The assignment brief requires a real corpus and labelled evaluation set, so placeholder figures must be replaced before submission.

TASK 11 – DESIGN TRADE-OFFS AND DEPLOYMENT

No normalisation is easy to maintain and has low computational cost, but it cannot reliably connect inflectional variants. Porter stemming is inexpensive and requires no large lexicon, but it may over-stem or under-stem specialised terminology. Full finite-state morphology offers greater control and interpretable analyses, but its coverage depends on the lexicon and rules.

A hybrid design is recommended because the hospital problem contains both fixed-format entities and open-ended language. Regex and DFA are appropriate for identifiers, dates and measurements. The FST handles known word forms and grammatical features. Porter stemming can be used as a fallback for unknown English-like words, but the result should be marked as approximate.

Search results should indicate the match type. An exact match should be distinguishable from an inflectional match, a derivationally related match and a fuzzy OCR correction. This allows clinicians to understand why a record was retrieved and to judge whether the relationship is relevant.

Maintenance effort includes vocabulary review, affix licensing, pattern updates, abbreviation governance, annotation and quality assurance. The design should include a change log and versioned resources. A new rule should be tested against the full regression suite before deployment.

A learned model may become more economical when there is enough labelled data and vocabulary changes faster than a manually curated lexicon can be maintained. However, learned predictions should not replace strict validation for critical identifiers, dates or medication strengths.

TASK 12 – OCR AND CODE-MIXED INPUT: EXPANDED IMPACT

OCR output may contain character substitutions, missing punctuation, broken words and line-wrap hyphens. A handwritten zero may be recognised as O, a one as lowercase l, and the sequence rn as m. A unit such as mg may be split into separate tokens. These errors affect both tokenisation and entity recognition.

Romanised regional-language notes may contain English clinical terms embedded in regional-language grammar. A token-level language identifier is more suitable than assigning one language to the whole note because the language can change within a sentence. The system should also allow a mixed label where a token contains elements from more than one language.

Weighted edit-distance matching can generate candidate corrections and assign costs to edits. Common OCR confusions can receive lower costs than unlikely substitutions. However, the nearest spelling is not always the correct clinical term. Candidate correction should be conditioned on field type, vocabulary and surrounding context.

Line-break repair should be conservative. A hyphen at the end of a line may be a formatting artefact or a genuine compound boundary. The original line and reconstructed token should both be retained. Unicode normalisation can standardise equivalent character representations before tokenisation.

The revised pipeline should preserve confidence values and provenance at each stage. An uncertain patient identifier or medication strength should be flagged for human verification. It is safer to leave a field unresolved than to silently create a plausible but incorrect clinical value.

APPENDIX – EXPERIMENTAL REPORTING TEMPLATE

Corpus summary: report the total number of records, total word tokens, number of records by type, source type, de-identification approach and date convention. State how synthetic records were created if synthetic data are used.

Entity evaluation: for every entity type, report TP, FP, FN, precision, recall and F1. Include two or more representative error examples and explain whether each error was caused by OCR, ambiguity, spacing, abbreviation, an unsupported format or a pattern-design issue.

Morphology evaluation: report the number of annotated word forms, number receiving at least one analysis, exact analysis accuracy, ambiguity rate and unknown-token rate. Include examples where the FST and Porter stemmer agree and examples where they diverge.

Retrieval evaluation: list the 40 queries or provide a query appendix, record the relevance judgements and report precision, recall and F1 for no stemming, Porter stemming and full morphological analysis. Use the same corpus and judgements for every condition.

Runtime evaluation: report machine specifications, Python version, NLTK version, corpus size, elapsed time and number of processed items for each stage. Include at least one plot and explain the trend rather than only inserting a graph.

Final quality check: ensure that all numerical results are measured or labelled illustrative, all code examples are executable, every table has a title, all abbreviations are defined and the original clinical text is preserved.